Listing 5.22 `SK_service_chat.py` (chat function)

```
async def chat() -> bool:
    try:
        user_input = input("User:> ")     #1
    except KeyboardInterrupt:
        print("\n\nExiting chat...")
        return False
    except EOFError:
        print("\n\nExiting chat...")
        return False

    if user_input == "exit":     #2
        print("\n\nExiting chat...")
        return False
    arguments = sk.KernelArguments(     #3
        user_input=user_input,
        history=("\n").join(
           [f"{msg.role}: {msg.content}" for msg in history]),
    )
    result = await chat_completion_with_tool_call(     #4
        kernel=kernel,
        arguments=arguments,
        chat_plugin_name="ChatBot",
        chat_function_name="Chat",
        chat_history=history,
    )
    print(f"AI Agent:> {result}")
    return True
```

#1 Input is taken directly from the terminal/console.
#2 If the user types exit, then exit the chat.
#3 Creates arguments to pass to the function
#4 Uses the utility function to call the function and execute the tool

Lastly, scroll down to the bottom of the file, and review the primary function. This is the code that calls the chat function in a loop.

Listing 5.23 `SK_service_chat.py` (main function)

```
async def main() -> None:
    chatting = True
    context = kernel.create_new_context()

    print("Welcome to your first AI Agent\     #1
\n  Type 'exit' to exit.\
\n  Ask to get a list of currently playing movies by genre."
    )
    while chatting:     #2
        chatting, context = await chat(context)     #3


if __name__ == "__main__":
    asyncio.run(main())
```

#1 Introduction to the user
#2 Continues until chatting is False
#3 Calls the chat function asynchronously